CAPSTONE DELIVERABLE 1

VarEx
Variance Explanation Assistant

Product Brief — a deterministic logic layer paired with a language layer, built so a finance analyst can explain a forecast miss in minutes without a model inventing the reason why.

Track: Track 4 — Finance

Prepared for: Capstone submission — Deliverable 1 of 2 (Product Brief)

Prepared by: Arun Agrawal

Date: July 2026

SECTION 01

Problem Statement & User Persona

USER PERSONA

Priya Chen — Senior FP&A Analyst, Retail Operations

Owns the monthly variance explanation for a portfolio of 45 store locations across multiple departments. Reports into a VP of Finance who reviews results live, store by store, in a monthly business review.

> *"I don't just need the number. I need to be able to say it out loud, in the room, and have it hold up when someone pushes back."*

The trigger moment. A store in her portfolio misses its target by double digits. Before the monthly review, leadership will ask the question that either ends a difficult conversation quickly or drags it out: *"Why did we miss the forecast?"* Priya has a narrow window to find out, and the honest answer is scattered across systems that don't talk to each other — a sales ledger, a weather feed nobody thought to check, a promotional calendar, a note about a competitor that opened two miles away, and her own memory of what happened in that store last year.

The core problem. Today, "explaining variance" usually means comparing actuals against a trailing rolling average of the store's own recent sales — a self-referential baseline that describes the recent past back to itself rather than an independent target set before the period began. On top of that shaky baseline, analysts manually eyeball a long list of contextual factors — weather, macroeconomic indicators, promotions, competitor activity, calendar shifts — to guess which ones actually mattered, then write a defensible narrative under time pressure. When that narrative oversimplifies or gets it wrong — attributing a miss to "soft consumer sentiment" when the real cause was a competitor opening two months earlier — it isn't just the number that takes the hit. It's Priya's credibility.

What "good" looks like. An independent, pre-computed forecast baseline that isn't just a mirror of recent actuals; every plausible driver checked systematically instead of from memory; a narrative that cites only what the math actually supports; and a confidence signal that tells Priya, honestly, when the explanation is solid versus when it deserves a caveat in the room.

SECTION 02

Job-to-be-Done (JTBD) Statement

> **When I** see variance in numbers,
> **I want to** explain what caused it clearly and quickly,
> **so I can** defend my analysis and maintain credibility with leadership.

Each clause sets a distinct product requirement, and the product has to satisfy all three simultaneously — optimizing for one at the expense of another defeats the JTBD:

- **"Clearly"** demands a fluent narrative a VP can absorb in seconds, not a table of z-scores. This is the argument for a language layer.
- **"Quickly"** demands the explanation exist before the review starts, not after — which rules out manual cross-referencing of five systems under time pressure.
- **"Defend... and maintain credibility"** is the hardest constraint: the explanation must survive scrutiny after the fact, not just sound plausible in the moment. This is the argument for a deterministic logic layer underneath the narrative — credibility is destroyed the first time a cited cause turns out to be invented.

SECTION 03

Why AI?

The JTBD's three demands can't all be met by one kind of system alone. Each failure mode below is why the product is deliberately split into a deterministic logic layer *and* a language layer, rather than being built as either alone.

Why a pure script fails

A rules engine can compute a z-score, flag a threshold breach, and output a classification label — and it should, since that part of the problem has one correct answer. But it cannot compose "the volume miss was compounded by a promotional pull-back mid-month, and partially offset by unseasonably strong week-one traffic" out of a handful of numbers. Synthesizing several simultaneous, competing drivers into the specific, situational prose a VP expects to hear in a live review is an open-ended language task — exactly what a rules engine is not built to do. A script can tell Priya *that* something moved. It cannot tell her the story leadership needs to hear.

Why a pure LLM fails

Asked directly to "explain why sales missed," an ungrounded LLM will confidently produce operationally plausible-sounding causes — "shifting consumer sentiment," "increased regional competition" — with no connection to what the data actually shows. This is the hallucination trap the assignment specifically calls out,

and it is not a cosmetic risk: a hallucinated cause in a leadership deck is a credibility-ending event for the analyst who presented it, not a minor factual error. A pure LLM also cannot be trusted to perform the arithmetic itself — variance percentages and z-scores computed by a language model are not reliably reproducible or auditable, which fails the JTBD's "defend my analysis" requirement outright.

Why the split is the right architecture

The deterministic logic layer computes and locks every number and every classification — Volume, Price, Timing, Force Majeure, Competitive Pressure, or Anomaly — before the model is ever called. The LLM's only remaining job is translating an already-verified set of facts into the fluent, context-aware sentence a human analyst would say out loud in a review. That is precisely the class of task language models are suited for, and precisely the class of task a rules engine is not — which is why this product needs both, in that order, and never lets the second one touch the first one's job.

SECTION 04

Feature List — Value vs. Effort Matrix

Features are grouped by expected analyst value against build effort, to make the sequencing decision explicit rather than implicit in the backlog.

HIGH VALUE · LOW EFFORT — BUILD FIRST

- Deterministic variance calc (absolute $ and %)
- Largest-contributor isolation
- Structured input form + validation gate
- rubric_bucket lookup (classification → rubric mapping)

HIGH VALUE · HIGH EFFORT — CORE INVESTMENT

- ARIMA (SARIMAX) forecast baseline, precomputed per store-department
- Expanded classification decision tree (8 values, incl. Force Majeure, Competitive Pressure)
- Confidence score with contradiction + coverage penalties
- Portfolio Anomaly Queue / admin audit dashboard

LOW VALUE · LOW EFFORT — FILL-INS

- Seasonal index (offline, week-of-year multiplier per department)
- Year-over-year comparison (live query, no new table required)

LOW VALUE · HIGH EFFORT — DEFERRED

- Full 2010–2012 ARIMA backtest across every historical month (vs. bounded May–Oct 2012 window)
- Live/real competitor intelligence feed (vs. disclosed synthetic table)

SECTION 05

Failure Mode Analysis (FMA)

Every failure mode below was designed against before backend build started, not discovered afterward — each has a specific, testable detection mechanism and a mitigation that runs independently of whether the LLM "cooperates."

FAILURE MODE	DETECTION MECHANISM	MITIGATION
Hallucinated causal narrative	Forbidden-topic scan on generated text for any driver with |z| < 1.5 not otherwise cited	Prompt boundary restricts the model to cited facts; confidence docked 25 pts per forbidden hit
Contradicted math (narrative states wrong direction or magnitude)	Automated parse compares the narrative's directional language against the computed sign	Confidence docked 20 pts per contradiction; recorded in system_flags for audit
Division-by-zero / zero forecast baseline	Pre-AI validation gate checks forecast_sales before any calculation runs	Routed to rolling-average fallback; flagged insufficient_baseline_history
Thin ARIMA history (<52 weeks before cutoff)	fit_cutoff_date + baseline_weeks_available check at seed time	Automatic fallback to trailing rolling average; system_flags entry; confidence coverage penalty (−10)
Competitor ambiguity (synthetic data muddying attribution)	competitor_impact_score ≥ 0.2 check	Confidence coverage penalty (−20) applied whenever present, even when competitor pressure isn't the headline driver
Normal seasonal swing misread as an anomaly	Classification runs on adjusted_variance_pct, never on raw variance	Seasonal index is applied before the decision tree ever sees the number
Rubric taxonomy mismatch (expanded classification vs. the rubric's literal four values)	Design-time risk, not a runtime failure	Parallel rubric_bucket field, computed via a fixed lookup, shown alongside classification in the dashboard as a hedge

SECTION 06

AI Trade-offs

Speed vs. Accuracy

Precomputing the ARIMA baseline offline — rather than fitting it live per request — trades the ability to do instant "what-if" reforecasting for perfect reproducibility. The same store, department, and month always resolves to the same baseline, which is what lets an analyst defend a number under scrutiny weeks later rather than getting a slightly different answer each time the report is regenerated.

Creativity vs. Determinism

The model is given zero creative latitude over classification, z-scores, or confidence — all three are locked before inference runs. Its creative latitude is scoped narrowly to sentence construction and tone within a fixed factual scaffold, at T = 0.0. This sacrifices narrative novelty — a "Force Majeure" report will always feel structurally similar to another — in exchange for trustworthiness. That is the correct trade for a finance product defending a number to leadership; it would be the wrong trade for a marketing or creative-writing product.

Model Simplicity vs. Fit Quality

A fixed-order SARIMAX(1,1,1) is used for every store-department series instead of a per-series auto_arima search. With only about 2.5 years of weekly history available, a full 52-week seasonal ARIMA is data-starved — reliably estimating it wants three or more seasonal cycles, and there are only two here. The simpler, fixed model is both cheaper to compute (roughly 3,300 fits in minutes, not hours) and more honest about what the underlying data can actually support than a heavier model would be.

SECTION 07

Cost & Sustainability Framework

- **Compute cost is front-loaded, not per-request.** The ~3,300 ARIMA fits happen once at seed time — or on a periodic reseed cadence — never on the live request path an analyst is waiting on. The expensive statistical work never touches response latency.
- **LLM cost is bounded and predictable.** Each report triggers exactly one short completion at T = 0.0 against a fixed-size structured prompt: verified JSON facts plus a short analyst note. There are no multi-turn agentic loops and no retries built into the design, so per-report token cost stays small and constant regardless of how complex the underlying variance turns out to be.
- **Volume is bounded by design, not by luck.** Roughly 3,300 real store-department combinations exist in the portfolio, but reports are generated on demand when an analyst investigates an item flagged by the Portfolio Anomaly Queue — not exhaustively, for every combination, every month. LLM call volume scales with actual flagged anomalies, not with total portfolio size.
- **Reports are cached, not regenerated.** variance_reports persists inputs, computed math, and the narrative together as one record. Re-opening the same store, department, and month does not trigger a second LLM call — sustainability here means idempotent storage as much as cheap inference.

SECTION 08

Retrospective Reflection

TO BE COMPLETED AFTER THE WORKING PROTOTYPE IS BUILT AND DEMOED

This section is intentionally left as a placeholder — a retrospective written before the build exists would be speculation, not reflection. Once the prototype and demo video are finished, this section should answer:

- What had to be simplified given time constraints (e.g., synthetic competitor data instead of a live feed, a bounded May–Oct 2012 ARIMA window instead of a full historical backtest) — and whether that simplification held up under demo scrutiny.
- What worked better than expected once real data was flowing through the logic layer versus what the architecture assumed on paper.
- Where the confidence score and forbidden-topic guardrails actually caught something during testing — a concrete example is stronger evidence than a description of the mechanism.
- What would change with more time: full historical ARIMA coverage, a real competitor intelligence integration, or a different classification taxonomy than the expanded one chosen here.

INTERIM FINDING — LOGGED MID-BUILD (2026-07-15)

Item 2 above asks what worked differently than the architecture assumed on paper — one instance surfaced already. **seasonal_index** was originally pooled across all 45 stores per department. Correlation analysis showed store *type* — not region, which is synthetic and non-causal in this dataset — is the real driver of seasonal variation between stores in the same department. The table and ETL step (**etl/04_seasonal_index.py**) were re-keyed to **(dept_id, store_type, iso_week)**; verified live that two stores of different types in the same department now produce different seasonal-adjustment values (migration **20260715120000_rekey_seasonal_index_by_store_type.sql**). Logged here so the finding isn't lost before Section 08 is written in full.